

Project 13 — Text to Speech

Fine-tuning **VITS** for end-to-end neural speech synthesis

Dima · Emir · Ilya

192.151 Introduction to Deep Learning · 2026S · TU Wien

The task

Build a system that maps text → audio, and evaluate it.

Choose **one** pipeline:

	Pipeline	Stages
A	FastSpeech2 (text→mel) + HiFi-GAN (mel→wav)	two models
B	Tacotron2 (text→mel) + neural vocoder	two models
C	VITS — end-to-end text → waveform	one model

Required: train on a held-out test set · report **epochs / batch size / GPU-hours** · qualitative gallery + failure modes · **evaluate** with ASR → **WER/CER** and a spectral metric (**MCD**).

| We chose **Option C — VITS**.

Our story in four decisions

1. **Pivoted A → C.** Started on SpeechT5 + HiFi-GAN (two models); moved to **VITS** — one end-to-end model, better audio out of the box.
2. **Fine-tune, not train from scratch.** Start from a strong LJSpeech checkpoint (`vits-ljs`) → new voice in a few GPU-hours, still *real* training.
3. **Built our own dataset — a "B1 Battle Droid" voice.** Makes the before/after **audible**, not just a number.
4. **Wrote the VITS training loop from scratch.** Hugging Face ships VITS *inference-only* — we re-implemented training, alignment, losses, and the GAN.

What is VITS?

One network, text → raw waveform. No separate vocoder, no intermediate mel.

```
text → [ text encoder + flow prior + stochastic duration  
         + posterior encoder (train only) + HiFi-GAN decoder ] → waveform
```

Four ideas combined (*Kim, Kong & Son, ICML 2021*):

- **Conditional VAE** — a latent `z` of the speech
- **Normalizing flow** — a more expressive prior over `z`
- **Adversarial learning (GAN)** — discriminators push toward realistic audio
- **Stochastic duration predictor** — one text → many valid rhythms

VITS components — train vs. inference

Component	Role	Train	Infer
Text encoder	text → prior (mean, log-var) over z	✓	✓
Posterior encoder	real-audio spectrogram → posterior z	✓	—
Flow	invertible map $z \leftrightarrow z_p$	✓	✓
Duration predictor	length of each text token	✓	✓
HiFi-GAN decoder	$z \rightarrow$ waveform	✓	✓
Discriminators (MPD+MSD)	real vs. fake (signal only)	✓	—

Asymmetry: in training the posterior sees *real* audio; the text-prior learns to match it (**KL**). At inference there's no audio → sample z from the prior.

The alignment problem → MAS

Text tokens and audio frames have **different lengths** and **no ground-truth alignment**.

Monotonic Alignment Search (MAS): a dynamic-programming search (Viterbi-like) for the highest-likelihood **monotonic** text → frame map.

```
tokens → H E L L O
frames → HH EEE L LLLL 00000 (many more frames)
           ← monotonic, never goes backward
```

We implemented MAS **from scratch** (`alignment.py`): a vectorized forward pass that accumulates best log-likelihood per (token, frame), then a backward traceback. `@torch.no_grad` — it's a *search*, not a learned layer.

The data story — a droid voice

- **Base: LJSpeech** — single English speaker, **13,100** clips (~24h).
- **Fine-tune target:** a custom voice **we built & published** → `Dmi1tr13/ljspeech-b1` on the Hugging Face Hub.

Why a "B1 Battle Droid" voice? The fine-tune's effect becomes **unmistakable to the ear** — a subtle voice makes a demo ambiguous.

Pipeline we built:

```
LJSpeech text → VITS synth → RVC voice conversion → resample 22.05 kHz → HF Hub  
(bake_dataset)                (B1 droid, Docker)      (prepare_b1)      (push_b1)
```

RVC changes **timbre, not pronunciation** → transcripts stay valid → instant 13,100-clip **paired** dataset.

The core challenge

Hugging Face's `VitsModel` is inference-only.

No training forward pass · no alignment · no losses · no discriminator.

To fine-tune it, we re-implemented the **entire** training stack on top of HF's modules — text encoder, posterior encoder, flow, decoder:

- `model.forward_train` — the training-mode forward pass
- `alignment.py` — Monotonic Alignment Search
- `losses.py` — five loss terms
- `discriminator.py` — HiFi-GAN MPD + MSD
- `train.py` — the alternating GAN training loop

This is the project's main engineering contribution.

forward_train — the training pass

For each batch:

1. **Text encoder** → prior mean, log-var
2. **Posterior encoder** (real spectrogram) → latent z
3. **Flow** $z \rightarrow z_p$
4. **MAS** → hard monotonic text → frame alignment, align prior to frames
5. **Duration loss** vs. MAS-derived durations
6. **Random segment slice** of z → **decoder** → predicted waveform
(*decoder is costly → train on a short crop, not the whole clip*)
7. Return waveform + mel, prior/posterior params → feed the losses

The loss = five terms

Term	What	Weight
Reconstruction	L1 on the mel (predicted vs. target)	45
KL	posterior $\parallel$ prior on the latent (the VAE term)	1
Duration	from the stochastic duration predictor	1
Adversarial	least-squares GAN (LSGAN) for G and D	1
Feature matching	L1 on the discriminator's feature maps	1

Reconstruction is weighted **45×** — getting the spectrum right matters most; the adversarial + feature-matching terms add realism and stabilize the GAN.

The discriminator (HiFi-GAN style)

Built from scratch — two complementary discriminators:

- **MPD — Multi-Period Discriminator**

reshapes 1-D audio into 2-D by periods (2, 7, 13, 29, 37, 73, 97, 113, 137) ;
catches **periodic structure** → pitch & harmonics.

- **MSD — Multi-Scale Discriminator**

1-D conv stacks at **3 scales** (audio progressively avg-pooled);
catches structure at **different time scales**.

Both expose **feature maps** → drive the feature-matching loss.

The GAN training loop

- Two **AdamW** optimizers — one for the generator (VITS), one for the discriminator.
- **Per step:**
 - **D step** — real vs. *detached* fake audio → discriminator loss
 - **G step** — recon + KL + duration + adversarial + feature-matching
- Checkpoints every N steps + end of epoch (`epoch_N_G.pt` / `_D.pt`),
resumable.
- Defaults: `batch_size=16` , `lr=2e-4` , `segment_size=16384` , 22.05 kHz, 80 mels.

Runs on free Colab GPU — runbook in `docs/training-on-colab.md` (checkpoints saved to Drive; Colab VMs are ephemeral).

Evaluation

Two complementary, objective metrics (lower = better):

- **WER / CER** — *intelligibility*. Whisper transcribes our audio → compare to the text (`jiwer`). "Can a listener understand it?"
- **MCD** — *spectral fidelity*. Mel Cepstral Distortion (`pymcd` , WORLD + DTW) vs. real audio. "Does it sound like the target?" (~5–8 dB typical.)

Before/after = one flag. `--checkpoint pretrained` vs. `models/finetuned` ;
same manifest feeds synthesis *and* scoring (a contract).

model	WER	CER	MCD
pretrained <code>vits-ljs</code> (baseline)	5.5%	1.6%	6.38 dB

Engineering — built to ship by a team of 3

- **Contract-first architecture.** Stages talk through tiny contracts in `core/` (a `{id, text, ref_audio}` manifest; an `EvalResult`), not internals.
→ parallel work, and the model stayed **swappable** (the A→C pivot barely touched anything downstream).
- **Reproducible:** pinned deps · `Makefile` one-liners · fixed seed.
- **Offline mock + smoke test** (`make all`) — prove the wiring in seconds, no GPU/downloads.
- **Quality gates:** `ruff` lint + format · `loguru` logging.

Status — honest snapshot

Component	State
Data + B1 dataset (published to HF Hub)	✓ done
VITS inference + checkpoint swap	✓ done
Fine-tuning loop (forward_train · MAS · 5 losses · MPD+MSD · GAN)	✓ implemented
Evaluation (WER/CER/MCD + before/after)	✓ done
Full GPU run + final fine-tuned metrics	⚠ pending

Everything is built and wired. The remaining step is the multi-hour GPU run that produces the fine-tuned-vs-pretrained numbers and the B1-voice demo.

Failure modes (and risks)

- **Mispronunciation** — rare words, names, numbers (char tokenizer, no phonemes) → motivates the optional phoneme study.
- **Prosody** — flat/odd rhythm; stochastic duration helps, isn't perfect.
- **Artifacts** — metallic/buzzing GAN artifacts, esp. early in training.
- **Fine-tuning risks** — too-high LR wrecks the pretrained weights · GAN instability · over-fitting the B1 timbre.

Demo

- **Pretrained VITS** on test prompts → `audio/generated/test_000*.wav`
- **B1 Battle Droid** target voice → `droid_test.wav`
- **Voice effects** (bonus) → `audio/demo_*.wav` — normal, accent, excited, ...

Live: `python -m vits_finetune.synthesize --checkpoint <ckpt> --text "Hello there."`

Team

Owner	Area
Dima	model & fine-tuning — <code>model/</code> , <code>vits_finetune/</code>
Emir	infra, contracts, data pipeline — <code>core/</code> , <code>data/</code> , <code>Makefile</code>
Ilya	evaluation — <code>evaluation/</code>

32 commits · 30 May → 18 Jun 2026

Takeaways

- **VITS** = one end-to-end model: CVAE + flow + GAN + stochastic duration.
- We **fine-tuned** a pretrained checkpoint onto a **custom voice we built and published** — adaptation in GPU-hours, audible result.
- We **re-implemented VITS training from scratch** on HF's inference-only model: MAS, five losses, HiFi-GAN MPD+MSD, the full GAN loop.
- **Contract-first** engineering let three people build in parallel and made the A→C model swap cheap.
- Evaluated with **WER/CER** (intelligibility) + **MCD** (spectral fidelity).

Thank you

Questions & discussion

github.com/Dmitri1S9/DL · 🙌 [Dmi1tr13/ljspeech-b1](#)